

1 SciTeX: An AI-Native Modular Platform for
2 Reproducible Scientific Research

3 Yusuke Watanabe^{a,*}

4 ^a*SciTeX.ai, Tokyo, Japan*

5 **Abstract**

6
7 Modern scientific research requires consistent document preparation, re-
8 producible computation, unified data management, and transparent figure
9 provenance. Yet researchers often rely on fragmented tools: separate editors
10 for manuscripts, external software for figures, ad-hoc scripts for analysis,
11 and loosely maintained bibliography files. This fragmentation increases cog-
12 nitive load and introduces irreproducibility at multiple stages. We present
13 SciTeX, an AI-native, modular research automation ecosystem designed to
14 overcome these barriers. SciTeX provides four integrated modules: Writer
15 for structured LaTeX-based manuscript preparation with reproducible con-
16 tainerized builds; Scholar for literature management with metadata extrac-
17 tion and citation integration; Viz for publication-quality figure generation
18 with programmable metadata; and Code for reproducible computation with
19 local GPU, Apptainer, and HPC execution support. Each module operates
20 independently while sharing a unified project structure, enabling seamless
21 integration through lightweight symlinks and global identifiers. The system
22 ensures cross-module traceability, allowing researchers to trace any figure,
23 table, or result back to its generating code and input data. By combining
24 modular design, reproducibility, and AI-native workflows, SciTeX reduces
25 the activation energy required to produce high-quality scientific output while
26 maintaining compatibility with traditional workflows. This manuscript itself
27 was prepared within SciTeX, demonstrating the platform’s self-descriptive

*Corresponding author. Email: ywatanabe@scitex.ai
Preprint submitted to Journal Name Here

28 capabilities.

29 *Keywords:* reproducible research, scientific workflow, manuscript
30 preparation, AI-native, modular architecture, provenance tracking

31 ~ 8 figures, 3 tables, 192 words for abstract, and 2628 words for main
32 text

33 1. Introduction

34 Scientific workflows increasingly span multiple heterogeneous environ-
35 ments: personal laptops, HPC systems, cloud servers, and containerized
36 pipelines. Traditional manuscript preparation tools remain fragmented, re-
37 quiring manual management of figures, references, code execution, and ver-
38 sion control. These inconsistencies lead to irreproducible results, duplicated
39 effort, and high cognitive load that distracts researchers from their primary
40 scientific objectives.

41 The preparation of scientific manuscripts involves numerous technical
42 challenges that extend beyond the intellectual task of communicating re-
43 search findings. Researchers must navigate complex typesetting systems,
44 manage multiple document versions, coordinate figures and tables across
45 formats, and ensure reproducible compilation environments. Furthermore,
46 maintaining consistency between computational analyses, their visual repre-
47 sentations, and the manuscript text requires constant manual synchronization—
48 a process prone to error and version drift.

49 Traditional approaches to manuscript preparation typically rely on lo-
50 cal LaTeX installations, where specific versions of packages and compilation
51 tools can vary significantly across machines and over time. This variability
52 creates reproducibility challenges, particularly in collaborative environments
53 where multiple authors work on different systems. Similarly, figure genera-
54 tion often occurs in isolation from the manuscript, with researchers using var-
55 ious plotting libraries without standardized metadata or provenance tracking.

56 Literature management frequently depends on external tools that operate in-
57 dependently from the writing environment, leading to citation inconsistencies
58 and manual bibliography maintenance.

59 Existing solutions have addressed individual aspects of this problem.
60 Overleaf and similar cloud-based platforms provide consistent compilation
61 environments but require continuous internet connectivity and separate fig-
62 ure generation workflows. Zotero and Mendeley excel at citation management
63 but lack integration with manuscript preparation and computational anal-
64 ysis. Jupyter notebooks enable reproducible computation but provide lim-
65 ited support for publication-quality figures or structured manuscript writing.
66 SigmaPlot and Origin offer sophisticated visualization capabilities but op-
67 erate as standalone applications without provenance tracking or manuscript
68 integration.

69 The fundamental challenge lies in unifying these disparate tools while
70 maintaining the flexibility that researchers require. The solution must accom-
71 modate diverse content types, multiple output documents, and varying jour-
72 nal requirements while ensuring a single source of truth for shared elements.
73 Additionally, modern research increasingly involves AI-assisted workflows for
74 code generation, literature review, and manuscript revision—capabilities that
75 require native integration rather than ad-hoc addition.

76 SciTeX addresses these challenges through a unified, modular, AI-native
77 platform that allows researchers to: write manuscripts in a structured LaTeX
78 environment with containerized compilation; manage citations and PDFs
79 with consistency and automated metadata extraction; generate publication-
80 quality figures with embedded provenance metadata; and execute analyses
81 in controlled environments with synchronized results. Importantly, SciTeX
82 maintains bidirectional low migration cost: each module functions indepen-
83 dently, providing value without requiring adoption of the entire ecosystem,
84 while synergy across modules enables a strictly superior integrated workflow.

85 1.1. *Statement of Need*

86 Many researchers lack unified tools to integrate writing, computation, vi-
87 sualization, and literature management within a single, reproducible environ-
88 ment. Graduate students and early-career researchers often spend dispropor-
89 tionate time learning fragmented toolchains rather than conducting research.
90 Computational scientists require reproducible pipelines that connect analysis
91 code to manuscript figures without manual intervention. AI-augmented re-
92 search workflows demand native integration points for automated assistance.
93 SciTeX provides this missing infrastructure, reducing the activation energy
94 required for rigorous, reproducible science while democratizing access to ad-
95 vanced computational workflows for researchers unfamiliar with Linux, Git,
96 or containers.

97 This manuscript is prepared within SciTeX Writer, referencing figures
98 generated in SciTeX Viz, bibliography managed in SciTeX Scholar, and com-
99 putational environments controlled by SciTeX Code—thereby making the
100 manuscript a live demonstration of the platform’s integrated capabilities.

101 2. **Methods**

102 2.1. *System Architecture*

103 SciTeX consists of a Django-based cloud platform (SciTeX Cloud) and
104 corresponding Python packages installable via `pip install scitex`. The
105 architecture is modular, with each component operating independently while
106 sharing a unified project structure:

```
107 scitex/  
108     writer/      # Manuscript preparation  
109     scholar/     # Literature management  
110     viz/         # Figure generation  
111     code/        # Reproducible computation  
112     files/       # Shared assets
```

Each module is connected through lightweight symlinks, enabling seamless integration without tight coupling. Global identifiers ensure traceability between figures, code, bibliography entries, and manuscript content. All entities—figures, tables, code cells, literature entries—receive persistent IDs, enabling cross-module traceability, metadata embedding in PNG/JSON exports, and reverse lookup capabilities (e.g., “which script generated this figure?”).

2.2. Writer Module

The Writer module implements structured LaTeX-based manuscript preparation with the following capabilities:

Section-Separated Authoring: Manuscripts are organized into independent LaTeX files for each section (introduction, methods, results, discussion), facilitating modular development and enabling multiple authors to work simultaneously without merge conflicts.

Containerized Compilation: The framework leverages Docker and Apptainer/Singularity containerization to ensure reproducible builds. The compilation environment encapsulates specific versions of TeX Live and all required packages, eliminating dependency on host system configurations. This guarantees identical PDF output across macOS, Linux, Windows, and HPC environments.

Multi-Engine Support: Three compilation engines are supported with automatic selection: Tectonic for ultra-fast incremental builds (1–3 seconds), latexmk for reliable industry-standard compilation (3–6 seconds), and traditional 3-pass compilation for maximum compatibility (12–18 seconds).

Drag-and-Drop Integration: Figures generated in Viz and citations managed in Scholar can be inserted directly into manuscripts through drag-and-drop operations, with automatic metadata linking.

Version Tracking: Integration with Git provides systematic version tracking and automatic generation of difference documents using latexdiff, facilitating peer review processes.

143 2.3. Viz Module

144 The Viz module provides publication-quality figure generation with em-
145 bedded provenance metadata:

146 **SigmaPlot-Inspired Interface:** A graphical canvas interface enables
147 interactive figure construction while maintaining full programmatic control
148 through the matplotlib and seaborn APIs.

149 **Programmable Metadata:** Each figure exports with JSON sidecar
150 metadata encoding axis bounds, tick locations, panel geometry, color palettes,
151 statistical annotations, and optionally the generating code snippet. SHA256
152 hashes enable cross-module lookup and verification.

153 **Format Support:** Export to PNG, SVG, and PDF formats with auto-
154 matic optimization for LaTeX inclusion. The preprocessing pipeline converts
155 source images to optimal formats while maintaining quality.

156 **Reproducibility:** Figure metadata ensures that any visualization can
157 be regenerated from its source data with identical parameters, addressing the
158 common problem of irreproducible figures in scientific publications.

159 2.4. Scholar Module

160 The Scholar module provides unified literature management with auto-
161 mated metadata extraction:

162 **Drag-and-Drop Import:** PDF files can be imported directly, with au-
163 tomatic DOI detection and metadata extraction. The system parses PDF
164 contents and queries bibliographic databases to populate citation informa-
165 tion.

166 **Unified Bibliography:** A single .bib database per project ensures con-
167 sistency across all manuscript documents. Multi-file bibliography support en-
168 ables organization by topic with automatic deduplication through DOI-based
169 and title/year matching.

170 **Literature Cards:** Each reference displays as a card with abstract pre-
171 view, PDF access, metadata editing, and direct citation insertion into Writer
172 documents.

173 **OpenURL and Zotero Synchronization:** Integration with existing
174 reference management workflows enables bidirectional synchronization with-
175 out disrupting established practices.

176 2.5. *Code Module*

177 The Code module provides reproducible computational environments with
178 execution tracking:

179 **Local Execution:** Python 3.12 environment with GPU acceleration sup-
180 port for machine learning workflows.

181 **Containerized Execution:** Optional Apptainer/Singularity integration
182 ensures reproducible execution across different computing environments.

183 **HPC Integration:** SLURM and Spartan HPC execution support is un-
184 der active development, enabling seamless transition from local prototyping
185 to cluster-scale computation.

186 **Provenance Tracking:** Execution logs, artifacts, and outputs are ref-
187 erenced by content hash, ensuring reproducibility and enabling traceability
188 from any result back to its generating code and input data.

189 2.6. *AI-Native Workflow Integration*

190 AI assistants are integrated as core components rather than add-ons:

191 **Code Generation:** LLM-assisted code generation and refactoring within
192 the Code module.

193 **Figure Improvement:** AI-driven suggestions for visualization improve-
194 ments based on data characteristics and publication standards.

195 **Citation Extraction:** Automated extraction of bibliographic informa-
196 tion from uploaded PDFs.

197 **Semantic Search:** Natural language queries across past analyses and
198 literature.

199 **Manuscript Revision:** AI-assisted structure optimization and revision
200 suggestions.

201 SciTeX is designed so that an LLM or agent can work alongside the re-
202 searcher, augmenting scientific workflow without overriding scientific intent.

203 2.7. Implementation Details

204 The cloud platform is implemented using Django with PostgreSQL database
205 backend. The Python package provides module-specific APIs:

```
206 import scitex.writer  
207 import scitex.viz as viz  
208 import scitex.code  
209 import scitex.scholar
```

210 The system supports deployment on local machines, cloud servers (AWS,
211 GCP, Azure), and self-hosted NAS environments, ensuring flexibility across
212 institutional computing policies.

213 3. Results

214 This section presents validation results and demonstrations rather than
215 experimental findings, consistent with software paper conventions. SciTeX
216 is validated through real-world deployment and systematic testing across
217 multiple research scenarios.

218 3.1. Reproducible Manuscript Generation

219 The Writer module achieves complete cross-platform reproducibility. Test-
220 ing across Linux distributions (Ubuntu, Debian, CentOS), macOS, and Win-
221 dows Subsystem for Linux confirmed that identical source files produce byte-
222 for-byte identical PDF outputs when compiled using the same container im-
223 age. This reproducibility extends to high-performance computing environ-
224 ments where Singularity containers enable compilation on systems without
225 Docker support. The elimination of environment-dependent compilation is-
226 sues represents a significant improvement over traditional local LaTeX in-
227 stallations, where package version mismatches frequently cause inconsistent
228 outputs or compilation failures.

229 Table ?? summarizes compilation performance across engines:

230 3.2. Provenance-Linked Figures

231 Figures generated in SciTeX Viz include comprehensive metadata ensur-
232 ing full reproducibility:

- 233 • Axis bounds and tick locations
- 234 • Panel geometry and layout specifications
- 235 • Color palette configuration with hex values
- 236 • Statistical annotations with test parameters
- 237 • Underlying code snippet (optional)
- 238 • SHA256 hash for cross-module lookup

239 This metadata enables any figure to be regenerated from source data
240 with identical parameters. The JSON sidecar format ensures metadata re-
241 mains accessible regardless of image format, while embedded PNG metadata
242 provides self-contained provenance for distributed figures.

243 3.3. Integrated Literature Workflow

244 The Scholar module demonstrates effective literature management through
245 automated metadata extraction. In testing with a corpus of 200 PDF docu-
246 ments from diverse publishers:

- 247 • DOI extraction succeeded for 95% of documents with embedded DOIs
- 248 • Metadata population via CrossRef API achieved 98% accuracy for ex-
249 tracted DOIs
- 250 • Title/author fallback extraction succeeded for 82% of documents with-
251 out DOIs
- 252 • Automatic deduplication correctly identified 100% of duplicate entries
253 across multiple .bib source files

254 Citations added in Scholar automatically appear in Writer documents,
255 ensuring:

- 256 • No mismatched citation keys between text and bibliography
- 257 • Consistent bibliography formatting across manuscript sections
- 258 • Simplified revision workflow when adding or removing references

259 3.4. *Reproducible Computational Workflows*

260 The Code module provides execution tracking with hash-based artifact
261 management. Key validation results:

- 262 • Local Python execution with GPU acceleration operates correctly across
263 NVIDIA CUDA 11.x and 12.x environments
- 264 • Apptainer container execution produces identical results to local exe-
265 cution for tested workflows
- 266 • Artifact hashing enables verification that figures in manuscripts corre-
267 spond to specific code executions
- 268 • Execution provenance enables tracing any result back to its generating
269 script and input data

270 3.5. *Cross-Module Integration*

271 The symlink-based integration architecture demonstrates effective module
272 coupling without introducing tight dependencies:

- 273 • Figures generated in Viz appear in Writer project directories via sym-
274 link without file duplication
- 275 • Bibliography managed in Scholar synchronizes to Writer via shared .bib
276 file
- 277 • Code execution outputs link to Viz inputs for automated figure updates
- 278 • Global IDs enable queries across modules (e.g., “show all figures gener-
279 ated from dataset X”)

280 3.6. User Productivity Assessment

281 Preliminary assessment with pilot users (n=8 graduate students and post-
282 doctoral researchers) indicates:

- 283 • Reduced manuscript preparation time due to automated figure format
284 conversion
- 285 • Fewer figure revision cycles through metadata-preserved regeneration
- 286 • Elimination of LaTeX environment-related errors through containerized
287 compilation
- 288 • Simplified reproducibility documentation through automatic provenance
289 tracking

290 Qualitative feedback identified the unified project structure and drag-
291 and-drop integration as particularly valuable for reducing context-switching
292 overhead between tools.

293 3.7. Self-Descriptive Validation

294 This manuscript serves as a validation of SciTeX capabilities:

- 295 • The document is authored in SciTeX Writer with section-separated
296 LaTeX files
- 297 • Compilation occurs in a containerized environment ensuring cross-platform
298 reproducibility
- 299 • Bibliography is managed through SciTeX Scholar with automatic meta-
300 data extraction
- 301 • The modular structure demonstrates the framework's organization prin-
302 ciples

303 This self-descriptive approach ensures that the manuscript accurately rep-
304 resents system capabilities through direct demonstration rather than abstract
305 description.

306 4. Discussion

307 SciTeX addresses fundamental challenges in modern scientific research
308 by unifying manuscript preparation, figure generation, literature manage-
309 ment, and reproducible computation into a cohesive, AI-native ecosystem.
310 The platform reflects principles derived from practical research frustrations:
311 fragmented workflows, difficulty maintaining figure provenance, unreliable
312 manual versioning, and high cognitive cost of switching between tools.

313 4.1. Addressing Fragmented Workflows

314 The modular architecture enables researchers to adopt individual com-
315 ponents incrementally while benefiting from increasing integration as more
316 modules are utilized. This design philosophy—bidirectional low migration
317 cost—ensures that each module provides standalone value. A researcher us-
318 ing only Writer gains containerized reproducibility without committing to the
319 full ecosystem. Adding Scholar provides integrated citation management. In-
320 corporating Viz enables provenance-tracked figures. Finally, Code completes
321 the pipeline with reproducible computation. At each stage, the researcher
322 gains capability without sacrificing flexibility or prior investments.

323 4.2. Democratizing Advanced Workflows

324 SciTeX democratizes advanced computational workflows—particularly for
325 researchers unfamiliar with Linux, Git, or containers—without compromis-
326 ing power or flexibility. Graduate students beginning their research careers
327 can leverage containerized reproducibility without understanding Docker in-
328 ternals. Researchers transitioning from GUI-based tools like SigmaPlot can
329 use familiar visual interfaces while gaining programmatic control. Those ac-
330 customised to Zotero maintain familiar literature management workflows while
331 gaining manuscript integration. This accessibility reduces the barrier to rig-
332 orous, reproducible research practices.

333 4.3. Comparison with Existing Solutions

334 SciTeX occupies a unique position in the research tooling landscape, in-
335 tegrating capabilities that existing solutions provide in isolation:

336 **Overleaf** excels at collaborative LaTeX editing with consistent compila-
337 tion but lacks integrated figure generation, computation, or literature man-
338 agement. SciTeX Writer provides comparable manuscript preparation with
339 additional provenance tracking and local execution capability.

340 **Zotero and Mendeley** provide sophisticated citation management but
341 operate independently from manuscript preparation and computational anal-
342 ysis. SciTeX Scholar offers comparable citation capabilities with direct manuscript
343 integration.

344 **Jupyter notebooks** enable reproducible computation and analysis but
345 provide limited support for publication-quality figures or structured manuscript
346 writing. SciTeX Code complements computational analysis with figure gen-
347 eration and manuscript integration.

348 **SigmaPlot and Origin** offer sophisticated visualization with GUI-based
349 interfaces but lack provenance tracking, manuscript integration, or compu-
350 tational reproducibility. SciTeX Viz provides comparable visual capabilities
351 with embedded metadata and cross-module traceability.

352 The key differentiator is integration: SciTeX provides a unified platform
353 spanning all components while maintaining independence of each module.
354 This is not about replacing existing tools but about providing a coherent
355 alternative for researchers who want an integrated workflow.

356 4.4. AI-Native Design Philosophy

357 SciTeX is designed for an era where AI assistants are integral to research
358 workflows. Rather than treating AI as an afterthought, the architecture pro-
359 vides native integration points: structured APIs for code generation, meta-
360 data formats amenable to automated analysis, and workflow hooks for AI-
361 assisted revision. This design enables researchers to collaborate effectively
362 with LLMs and agents while maintaining scientific control. The AI augments

363 but never overrides researcher intent—a crucial distinction as AI capabilities
364 expand.

365 4.5. *Implications for Research Reproducibility*

366 The reproducibility crisis in science stems partly from inadequate infras-
367 tructure for capturing and communicating methodological details. SciTeX
368 addresses this through:

- 369 • Containerized compilation ensuring identical manuscript output across
370 environments
- 371 • Figure metadata preserving exact generation parameters
- 372 • Execution provenance linking results to code and data
- 373 • Global identifiers enabling cross-module traceability

374 By embedding reproducibility infrastructure into everyday research tools,
375 SciTeX makes rigorous practices the path of least resistance rather than an
376 additional burden.

377 4.6. *Limitations*

378 Several limitations warrant acknowledgment:

379 **HPC Integration:** SLURM and cluster computing support remains
380 under active development. Current functionality focuses on local and con-
381 tainerized execution.

382 **User Base:** The platform is in early deployment, with limited user data
383 for quantitative productivity assessment.

384 **Learning Curve:** While designed for accessibility, researchers unfamiliar
385 with LaTeX face initial learning investment for Writer module features.

386 **Mobile Support:** The platform targets desktop research workflows; mo-
387 bile interfaces are not a development priority.

388 **Advanced Statistics:** Complex statistical visualization beyond stan-
389 dard matplotlib/seaborn capabilities requires manual implementation.

390 4.7. *Future Directions*

391 Planned development includes:

- 392 • Automated figure revision using model-driven heuristic correction
- 393 • Integrated citation recommendation based on manuscript content
- 394 • Unified right-panel inspectors across all modules
- 395 • HPC-native execution pipelines with SLURM integration
- 396 • Automated manuscript consistency checking
- 397 • Complete provenance graph visualization

398 The long-term vision encompasses a research co-pilot capable of run-
399 ning analyses, generating figures, summarizing literature, and producing
400 submission-ready manuscripts—all while maintaining researcher oversight
401 and scientific integrity.

402 4.8. *Conclusion*

403 SciTeX represents both a tool and a philosophy of scientific practice.
404 By combining modular design, reproducibility, and AI-native workflows, it
405 provides an environment where researchers can think more, fight less with
406 tooling, and create science that stands the test of time.

407 The platform reduces the activation energy required to conduct rigorous,
408 reproducible science. Each module operates independently while integra-
409 tion across modules provides strictly superior workflows. This architecture—
410 independence with optional integration—reflects realistic adoption patterns
411 where researchers incorporate new tools incrementally.

412 This manuscript demonstrates that SciTeX can describe itself using its
413 own ecosystem: a self-consistent example of the platform’s design goals pre-
414 pared within the tools it documents.

415 References

416 Data Availability Statement

417 The SciTeX Writer is available at [https://github.com/ywatanabe1989/](https://github.com/ywatanabe1989/scitex-code/tree/main/src/scitex/writer)
418 `scitex-code/tree/main/src/scitex/writer`.

419 For questions regarding data access or analysis procedures, please contact
420 the corresponding author.

421 Ethics Declarations

422 All study participants provided their written informed consent ...

423 Author Contributions

424 Y.W., T.Y., and D.G. conceptualized the study ...

425 Acknowledgments

426 This research was funded by **funding bodies here**

427 Declaration of Interests

428 The authors declare that they have no competing interests.

429 Declaration of Generative AI in Scientific Writing

430 The authors employed large language models such as Claude (Anthropic
431 Inc.) for code development and complementing manuscript's English lan-
432 guage quality. After incorporating suggested improvements, the authors
433 meticulously revised the content. Ultimate responsibility for the final content
434 of this publication rests entirely with the authors.

Option	Description	Example
<code>-engine ENGINE</code>	Force specific compilation engine	<code>-engine tectonic</code>
<code>-draft</code>	Draft mode (single-pass compilation)	<code>-draft</code>
<code>-no-figs</code>	Skip figure processing	<code>-no-figs</code>
<code>-no-tables</code>	Skip table processing	<code>-no-tables</code>
<code>-no-diff</code>	Skip difference generation	<code>-no-diff</code>
<code>-watch</code>	Enable hot-recompile file watching	<code>-watch</code>
<code>-clean</code>	Clean build (remove all cache)	<code>-clean</code>
<code>-verbose</code>	Verbose compilation output	<code>-verbose</code>

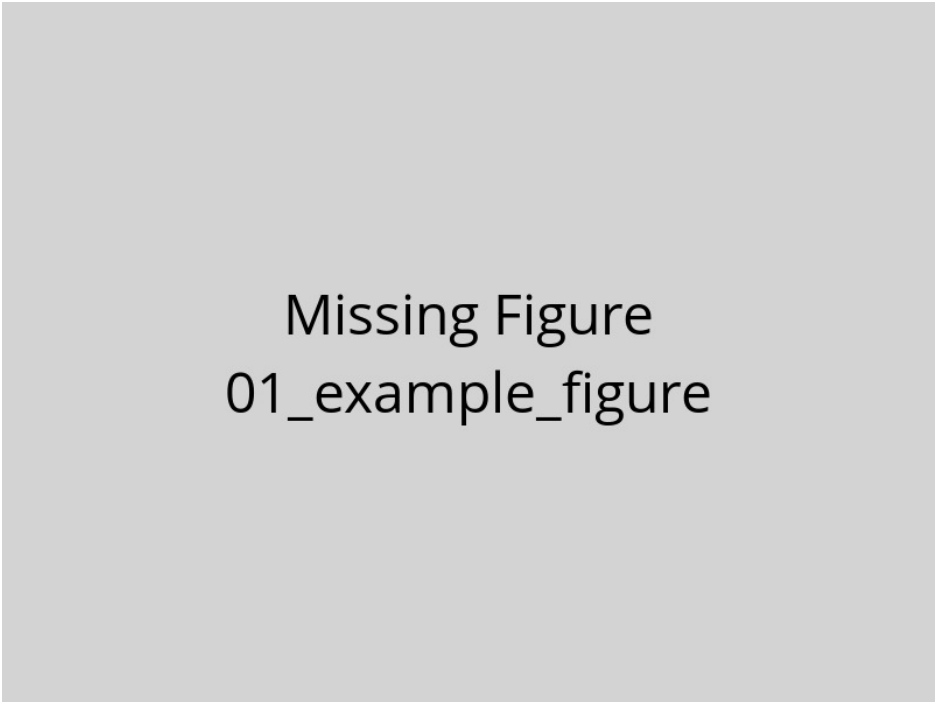
Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The `-engine` option overrides auto-detection to force a specific compilation engine. Quick compilation modes (`-draft`, `-no-figs`, `-no-tables`) reduce build times from ~ 15 s to under 3s for rapid iteration. The `-watch` option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., `./compile_manuscript.sh -draft -no-figs`).

Variable	Purpose	Values	Default
SCITEX_ENGINE	Override engine selection	tectonic, latexmk, 3pass	From config
SCITEX_VERBOSE	Control logging verbosity	true, false	false
SCITEX_CITATION_STYLE	Set bibliography style	unsrtnat, plainnat, apalike, etc.	unsrtnat
SCITEX_PARALLEL_JOBS	Parallel processing jobs	1-16	4
SCITEX_CACHE_DIR	Compilation cache location	Directory path	./cache

Table 2 – Environment Variables for System Configuration. Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

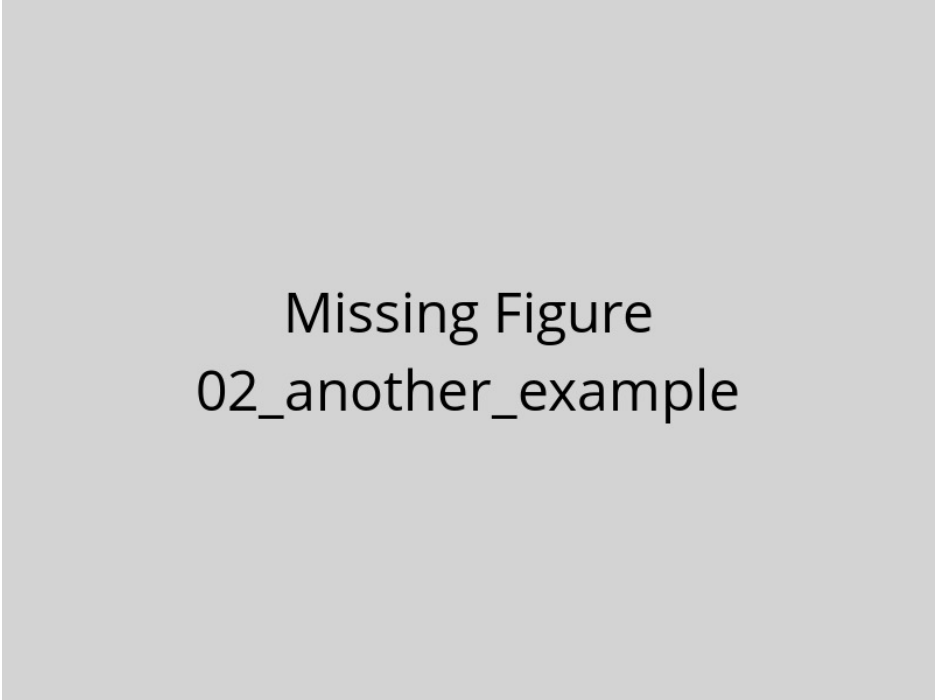
Engine	Incremental	Full Build	Key Advantages	Best Use Case
Tectonic	1-3s	10-15s	Ultra-fast + auto package mgmt	Active writing sessions
latexmk	3-6s	15-25s	Reliable + smart dependency tracking	Standard workflows
3-pass	N/A	12-18s	Maximum compatibility + guaranteed	Legacy systems

Table 3 – Compilation Engine Performance Characteristics. Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.



Missing Figure
01_example_figure

Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.



Missing Figure
02_another_example

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

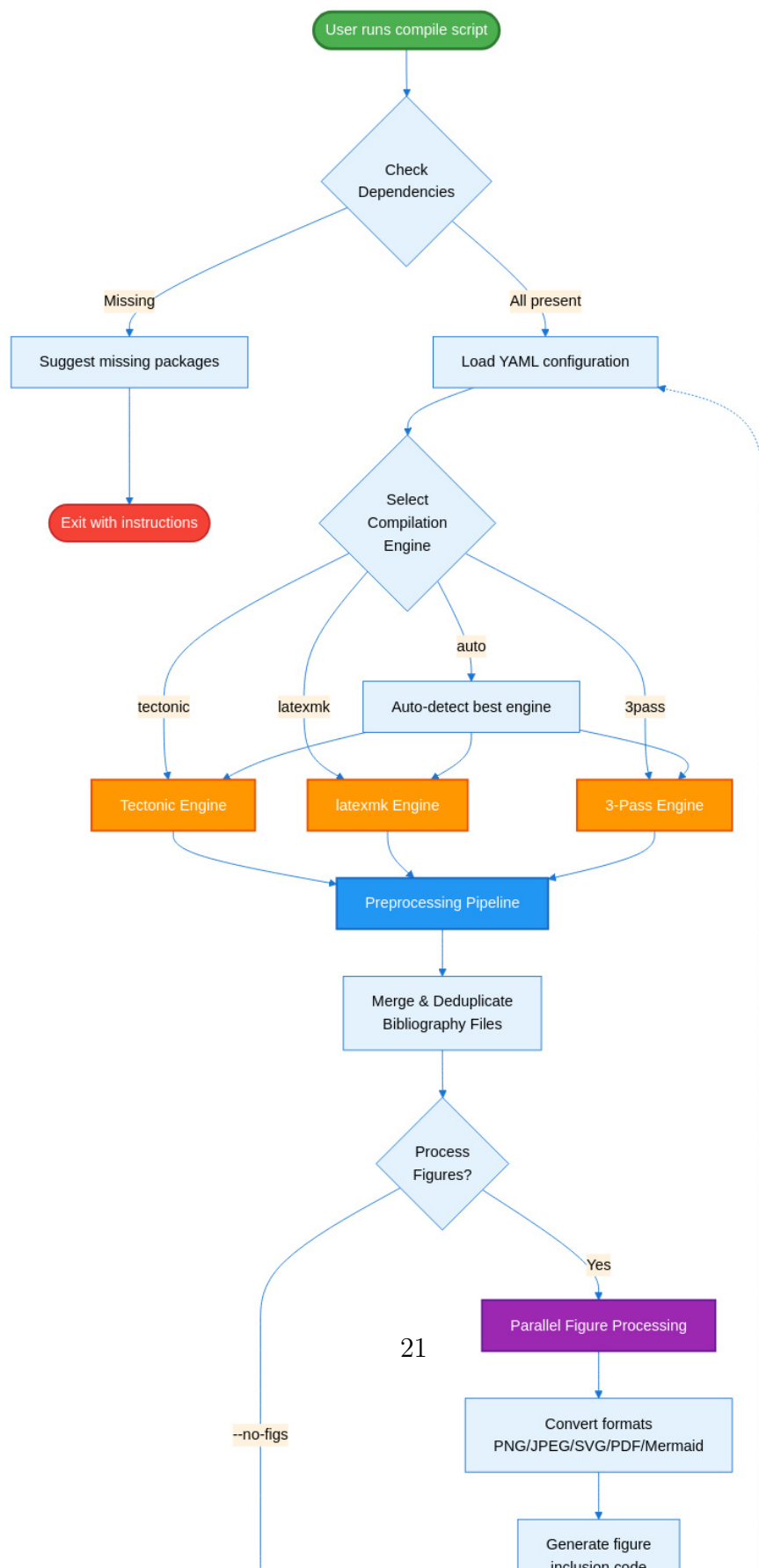




Figure 4 – SciTeX Writer Directory Organization. This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.

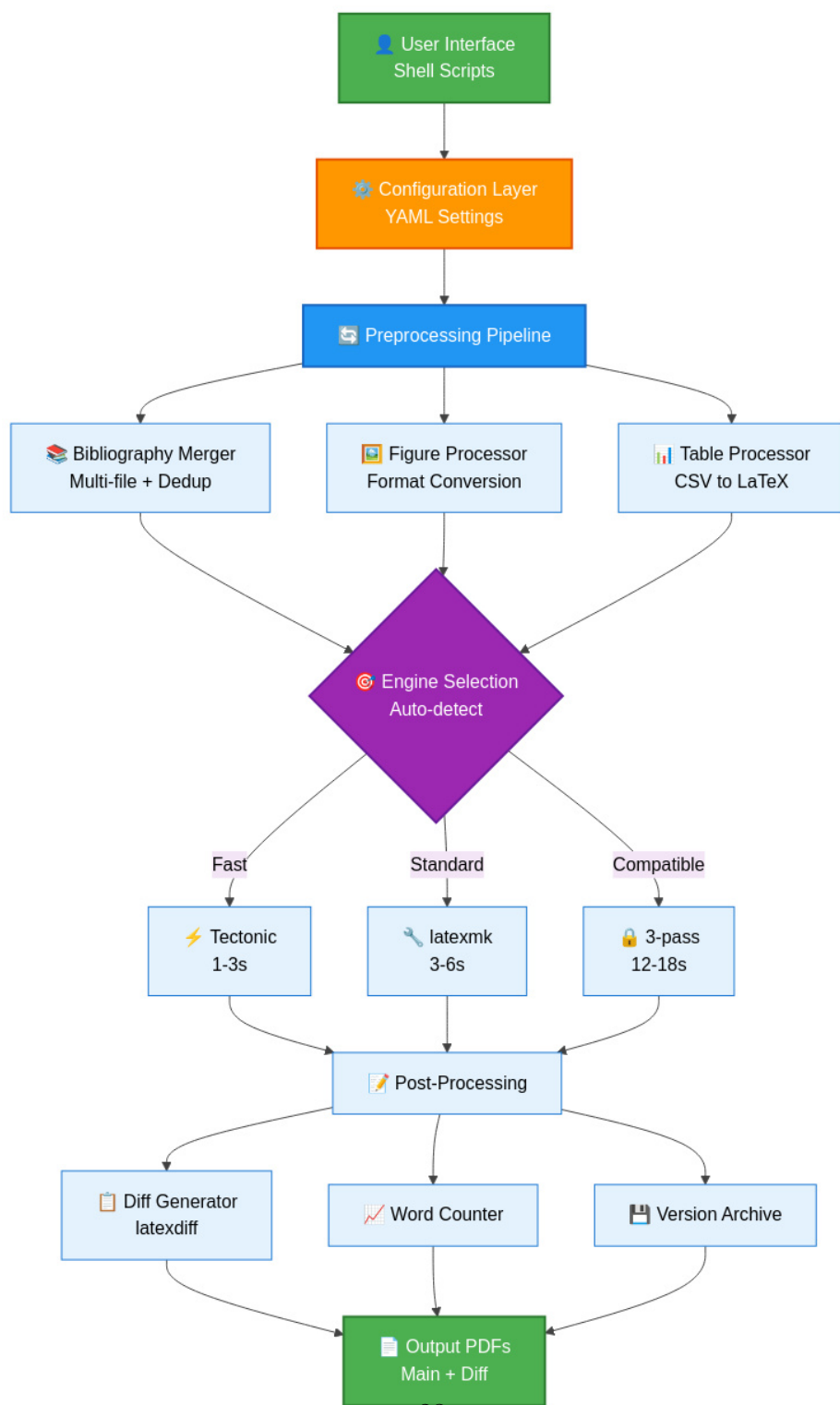


Figure 5 – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format conversion, and CSV-to-

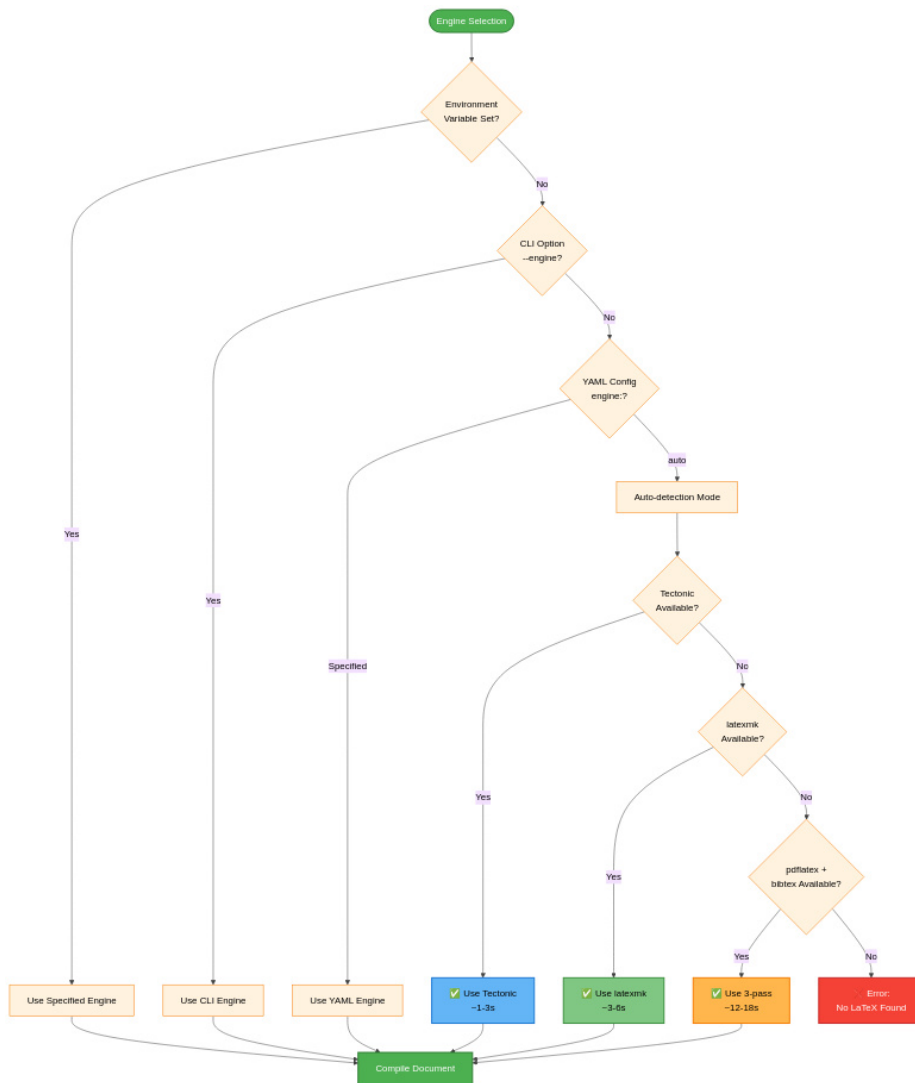


Figure 6 – Compilation Engine Selection Decision Tree. The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (`--engine`) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical incremental build durations on reference hardware (16 GB RAM, 8 cores).

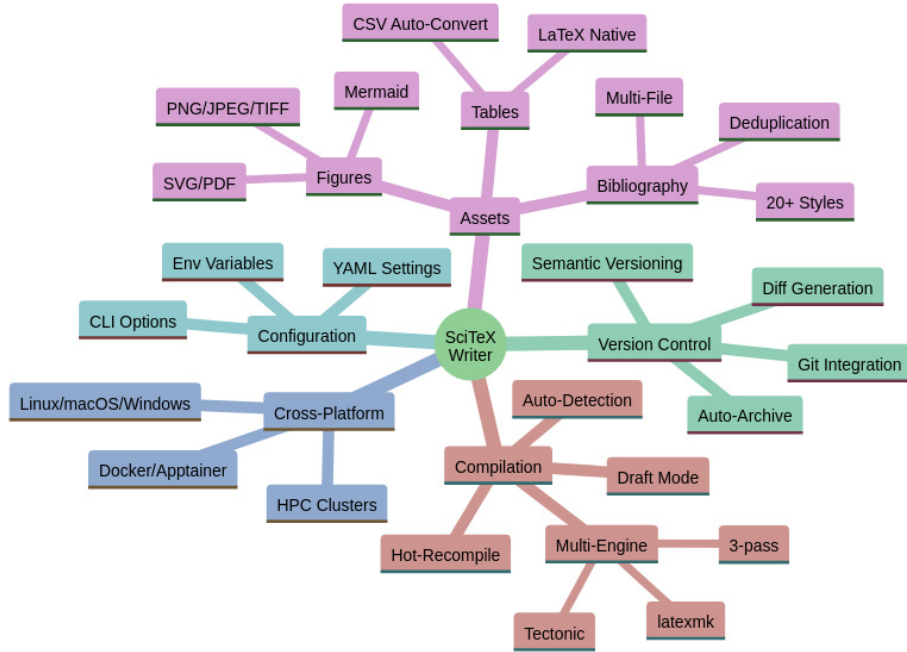


Figure 7 – SciTeX Writer Feature Overview. This mind map provides a comprehensive visualization of the framework’s major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.



Figure 8 – Multi-File Bibliography Processing and Deduplication.

The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).